Task 4-Pattern Justification

Patterns Identification

	Factory Method
	Builder
	Composite
	Iterator
	State
	Decorator
	Strategy
	Observer
	Mediator
	Chain of Responsibility
	Adapter
	Memento
	System classes (Not Part of GoF Patterns)
	Classes That extend the system (They are NOT PART OF IT). Added for the specific type of scenario

Keywords

- **Engine** = the workflow management system (TIVIDY). It is generic across business workflows and knows nothing about any one process.
- **Business workflow** = a case-based process in which people and outside systems complete work items, and approvals, rules and deadlines decide the route.

- **Work item** = an individual piece of work moving through a workflow, like a task that needs to be completed (claim verification, scoring an application, creating an account). In the class diagram this is (Input).
- **Workflow definition** = the blueprint, template or recipe of a process. It defines which steps exist, what order they run in, their rules, and who performs them. In the diagram this is (WorkflowDefinition), which holds a (ProcessDirector) (the recipe).
- **Workflow instance** = one actual, running execution of that blueprint (for example, Claim #1042 currently underway). In the diagram this is (WorkflowInstance).
- **Scenario layer** = the example processes that run on the engine (car insurance claims, postgraduate admissions, employee onboarding). They are not part of the engine, and the diagram keeps them in a separate grey package.

The examples below come from those three processes. Every class named in the participants lists is part of the generic engine unless stated otherwise.

Architectural Design Patterns

1. Factory Method

- **Problem in TIVIDY:** Different processes need different kinds of work items (a human task, an automatic step that calls an outside system). If the engine created each kind with new Concretex() everywhere, it would have to know every kind, and adding a new kind would mean editing the engine, which violates the Open/Closed Principle.
- **Participants:**
    - (InputCreator) (Creator, declares the factory method)
    - (HumanTaskCreator), (AutomatedTaskCreator) (ConcreteCreators)
    - (Input) (Product)

    - (HumanTask), (AutomatedTask) (Concrete Products)
- **Collaboration:** The engine asks an (InputCreator) for an (Input). The concrete creator decides which kind to make and returns it as an (Input), so the engine only sees the general type (polymorphism).
- **What varies:** The kind of work item that gets created. Even though the factory method returns an (Input), each kind does its work differently (a person completes one, an outside system completes the other).
- **Why appropriate:** The engine has to stay generic. The factory method lets a new process bring its own kind of work item without any change to the engine code.

2. Builder

- **Problem in TIVIDY:** A workflow has many parts, and each part has many settings: names, deadlines, dependencies, who may do it, approval needs. Setting everything in one giant constructor is error-prone and hard to read, and the steps differ between processes. A workflow definition also has to produce many independent running copies without sharing state between them.
- **Participants:**
    - (ItemManager) (Builder)
    - (ConcreteItemManager) (ConcreteBuilder)
    - (ProcessDirector) (Director)
    - (ClaimProcessDirector), (AdmissionsProcessDirector), (OnboardingProcessDirector) (scenario layer, ConcreteDirectors)
    - (Director) (Product: the finished tree of stages and work items)
- **Collaboration:** The ProcessDirector runs the construction steps in a fixed order (add a stage, add an item, set its deadline, set its approval chain, add a dependency). (ConcreteItemManager) carries out each step, using an (InputCreator) to make each work item, and then hands back the finished tree. The (WorkflowDefinition) keeps the (ProcessDirector) as its recipe, so starting a

new instance means running that recipe again. Every instance gets a freshly built tree, so changing one claim cannot change another or the definition.

- **What varies:** The settings, and the way a workflow is assembled for each process.
- **Why appropriate:** A workflow definition is assembled in many steps, and the same steps can produce different results for different processes. The builder also gives each instance its own copy of the structure, which is exactly the definition-versus-instance split the spec asks for.

3. Composite

- **Problem in TIVIDY:** Work is not flat. A process has stages, which contain tasks, which may also contain sub-processes. The engine should be able to start, complete or cancel a single task or a whole stage the same way, without checking which it is.
- **Participants:**
  - (Director) (Component)
  - (Input) (Leaf: a single work item)
  - (AggregateCollaborators) (Composite: a stage or sub-process holding children)
- **Supporting classes:** (Dependency) (links two parts, with an optional Condition), (CompletionRule) (decides when a stage is finished), (ExecutionMode) (sequential or parallel).
- **Collaboration:** A stage passes each request down to its children. Leaves do the real work. A stage is finished when its (CompletionRule) is satisfied, and it decides which child may start by checking its (Dependency) objects. A dependency can link two parts that sit in different stages, which is needed when, for example, a laptop must wait for an account created in another stage.
- **What varies:** The depth and shape of the work structure, and when a stage counts as complete.

- **Why appropriate:** This is exactly the "stages, work items and sub-processes" structure in the spec. It also covers each of the three example processes without change: parallel investigation (claims), a committee where three of five scores are enough (admissions), and IT, Facilities and Payroll setting up a new hire at the same time (onboarding).

4. Iterator

- **Problem in TIVIDY:** The engine has to go through a group of parts one by one, for example to find the next available work item, to collect progress, or to find overdue items, without depending on how the group stores them.
- **Participants:**
  - (Director) (Aggregate, creates iterators)
  - (Iterator) (Iterator)
  - (ReadyIterator), (DepthFirstIterator) (ConcreteIterators)
- **Collaboration:** A stage creates an iterator. The engine asks it for the next element until it is finished. (ReadyIterator) returns only the items that are ready to start. (DepthFirstIterator) visits every part, and is used by the deadline check and by reports.
- **What varies:** How the group is stored, and which items are visited in what order.
- **Why appropriate:** The spec asks for navigation of nested work "without requiring the rest of the system to know their internal storage structure". The same structure is walked in different ways for different jobs, and each way is its own iterator.

5. State

- **Problem in TIVIDY:** A work item changes over time (created, available, assigned, in progress, awaiting approval, completed, rejected, escalated, cancelled), and the actions that are valid depend on where it currently is. For example, a

cancelled item must not be completable. Checking a status field with large if/else chains is not allowed by the spec's rules.

- **Participants:**
  - (Input) (Context)
  - (InputState) (State)
  - (CreatedState), (AvailableState), (AssignedState), (InProgressState), (AwaitingApprovalState), (CompletedState), (RejectedState), (ApprovalState), (CancelledState) (ConcreteStates)
- **Collaboration:** The work item hands each action to its current state object. The state either allows the action and moves the item to the next state, or refuses it. (Completed), (Rejected) and (Cancelled) are final states: a rejected item is not reopened, and if the work has to be redone, a new follow-up item is created (see Mediator).
- **What varies:** The lifecycle rules: which states exist and which actions each state allows.
- **Why appropriate:** Each rule lives in the state that owns it, so adding a state (such as escalated) does not touch the others.

6. Decorator

- **Problem in TIVIDY:** A work item may need extra behaviour (priority handling, validation, auditing) without becoming a different kind of item. A subclass for every combination would multiply very quickly.
- **Participants:**
  - (Input) (Component)
  - (HumanTask), (AutomatedTask) (ConcreteComponents)
  - (Configuration) (Decorator, extends (Input) and holds the wrapped Input)
  - (PriorityConfiguration), (AuditConfiguration), (ValidationConfiguration) (ConcreteDecorators)

- **Collaboration:** A decorator wraps a work item, does its extra step before or after passing the call to the item it wraps, and decorators can be stacked.
- **What varies:** Which extra behaviours a given item has.
- **Why appropriate:** The spec names this exactly. For example, the oldest waiting claim, or the equipment order for a new hire who starts tomorrow, can be wrapped with priority handling and no new type of item is needed.

7. Strategy

- **Problem in TIVIDY:** Organisations make different decisions about the same kind of work. One assigns the next task by role, another by workload. One stage is finished when everything is done, another when three of five reviewers have scored. One route is followed only when a value is over a limit. These rules must not require the work item classes to be rewritten each time the process changes.
- **Participants:**
    - Input (Context for assignment), AggregateCollaborators (Context for completion), Dependency (Context for routing)
    - AssignmentStrategy (Strategy) with RoleBasedStrategy, LeastLoadedStrategy (Concrete Strategies)
    - CompletionRule (Strategy) with AllComplete, QuorumComplete
    - Condition (Strategy) with ThresholdCondition, FlagCondition
- **Collaboration:** The context holds a strategy object and asks it for the answer: who should get this item, whether this stage is finished, or whether this path should be followed. To avoid tight external coupling, the context (Input) passes itself into the strategy's algorithm via assign(context: Input*). This allows the strategy to operate directly on the data stored within the context object. The context itself does not know how the final answer was calculated.
- **What varies:** The assignment, completion, and routing algorithms.

- **Why appropriate:** Routing is needed in all three example processes: fraud flagged (claims), funding requested (admissions), security clearance needed (onboarding). Without a strategy object, every one of those variations would become a massive if/else chain inside the engine, which the system specification strictly forbids.

8. Observer

- **Problem in TIVIDY:** When something happens to a piece of work (it is completed, rejected, escalated or goes overdue), several other parts need to be informed: the next work item must become available, people must be told, and the history must be updated. If the work item called each of them directly, it would be tied to all of them and would have to keep track of every one.
- **Participants:**
    - (Input) (Subject)
    - (CompositeNotifier) (Observer)
    - (DependencyNotifier), (ParticipantNotifier), (HistoryNotifier) (ConcreteObservers)
    - (WorkflowEvent) (the information passed to observers)
- **Collaboration:** Notifiers register with an (Input). When the (Input) changes state, it informs every registered notifier, and each one reacts in its own way: one opens the next items, one tells the people involved, one records history.
- **What varies:** Which parts are listening, and what each one does.
- **Why appropriate:** The spec says completing one activity may make another available, an external event may trigger new work, and several parts may need to react to the same event. This is one-to-many, which is what Observer is for.

9. Mediator

- **Problem in TIVIDY:** The engine has many cooperating parts (notifiers, the builder, approval levels, the deadline monitor). If each called the others directly, every part would depend on every other, and a new part would mean changing many classes.
- **Participants:**
    - (Communication) (Mediator)
    - (ConcreteCommunication) (ConcreteMediator)
    - (Collaborators) (Colleague)
- **Colleagues:** (CompositeNotifier) and its subclasses, (ItemManager), (Authorizer), (DeadlineMonitor)
- **Collaboration:** Parts send messages only to the mediator, which decides who needs to hear about them. For example, when an approval level rejects a request, it tells the mediator, which asks the item manager to create a follow-up item for the rework and tells the notifier to inform the people involved. When the deadline monitor finds an overdue item, it tells the mediator, which tells the item to escalate.
- **What varies:** Who talks to whom.
- **Why appropriate:** The spec asks that these concerns be separated from the core workflow model so that new rules, interested parties or integrations can be introduced without redesigning the entire engine.
- **How this differs from Observer:** Observer is a broadcast: a work item announces its own change to anyone who subscribed. Mediator is coordination: engine parts that should not know about each other send their requests through one place.

10. Chain of Responsibility

- **Problem in TIVIDY:** Approval may involve one person, several levels, or escalation when nobody responds. Different processes have different levels and limits, and these should not be hard-coded into the work item.
- **Participants:**
    - (Authorizer) (Handler)
    - (DynamicAuthorizer) (ConcreteHandler, configured with a role, an approval limit and a timeout)
    - (ApprovalRequest) (the request that travels along the chain)
- **Collaboration:** A request goes to the first authorizer. If it is allowed to decide (the amount is within its limit), it does. If not, or if it does not respond in time, it passes the request to the next one in the chain.
- **What varies:** The number of levels, their order, and their limits.
- **Why appropriate:** The chain is assembled per process, so a new level is a new link and nothing else changes. The engine contains no job-title classes. Claims Handler, Claims Manager and Claims Director in the claims process, or Line Manager and HR Director in onboarding, are just DynamicAuthorizer objects given different roles and limits.

11. Adapter

- **Problem in TIVIDY:** Workflows must talk to outside systems whose interfaces were not designed for TIVIDY (a payment service, a credential-check service, a payroll system). The engine should not depend on any one vendor's interface.
- **Participants:**
    - (Input) (Target)
    - (AutomatedTask) (Adapter)
    - (Adaptee) (Adaptee: the outside system's own interface)

- **Scenario layer adaptees:** (PolicySystem), (FraudScoringService), (PaymentGateway), (CredentialCheckService), (StudentRecordSystem), (EmployeeRecordSystem), (AccountDirectory), (PayrollSystem)
- **Collaboration:** The engine uses the general (Input) interface. The adapting (AutomatedTask) translates those calls into the outside system's own calls (the (Adaptee)) and translates the answers back.
- **What varies:** The outside system being connected.
- **Why appropriate:** Connecting a new system just means writing a new adaptee and giving an automated task a reference to it. There are no changes to the engine.

12. Memento

- **Problem in TIVIDY:** The system must be able to undo or roll back when something goes wrong, for example when an action is reversed or a payment fails after earlier steps. Rolling back needs the earlier state, but outside classes should not be able to read or edit a workflow's internals.
- **Participants:**
    - (Director) (Originator: creates and restores snapshots)
    - (WorkflowSnapshot) (Memento)
    - (SnapshotHistory) (Caretaker)
- **Collaboration:** Before a change, the originator creates a snapshot of its state (every item's state, assignee and data). The caretaker stores it without looking inside. To undo, the caretaker hands the snapshot back and the originator restores itself.
- **What varies:** What is saved, and how many snapshots are kept.
- **Why appropriate:** The spec asks for retained history and rollback where sensible, and this keeps the saved state hidden.

Core architectural classes

ROLE

What this does for the system exactly

- It decouples tasks from specific humans. Instead of assigning a task to "John", you assign it to the "Reviewer" Role

PARTICIPANT

What this does for the system exactly

- It represents a physical human user logging into TIVIDY. It holds their identity and the list of Roles they are permitted to perform

PARTICIPANT REGISTRY

What this does for the system exactly

- It acts as the central employee database. When an AssignmentStrategy needs to find who should do a task, it asks this registry

INSTANCE STATUS

What this does for the system exactly

- It provides a global flag so the system instantly knows if an entire case is still active, finished, or aborted without searching every task.
- Not a class but an enumeration instead

enum class InstanceStatus { RUNNING, COMPLETED, CANCELLED, FAILED };

WORKFLOW DATA

What this does for the system exactly

- It serves as the "file folder" for a specific case. As the workflow moves between different tasks and external adapters, they read and write their variables (like payment amounts) to this central dictionary.

Basically uses a map to store the data, like (“approved”: “true” , “amount”: “15 000”) and this will be used to track a claim throughout its lifecycle

WORKFLOW DEFINITION

What this does for the system exactly

- It acts as the immutable master blueprint for a process. It holds the Builder recipe, ensuring that every time a new case starts, it builds an identical, fresh copy of the task tree.
- Basically a template for a process. So we pre build a process and store it in **DEFINITION REPOSITORY** so that we can use the template later instead of creating it again

DEFINITION REPOSITORY

What this does for the system exactly

- It acts as the filing cabinet for blueprints. When the engine needs to start "Admissions_v2", it retrieves the correct definition from here.

WORKFLOW INSTANCE

What this does for the system exactly

- It is the live, running execution of a single case. It binds the specific WorkflowData, the overall Status, and the freshly built task tree (Director) together in one package.

WORKFLOW ENGINE

What this does for the system exactly:

- It is the main API gateway. External UI buttons or REST APIs do not talk to tasks directly; they send commands here, and the engine routes them to the correct running **Instance**.